

Resumen — Fastbook 03

Arquitectura de aplicaciones web: conceptos clave

Este documento resume los conceptos clave del fastbook, centrado en la arquitectura de aplicaciones web: servidores, servidores proxy, servidores web y de aplicaciones, Kubernetes como orquestador de contenedores, y el patrón de diseño MVC. Autor original: Jaime Morales.

1. Servidor

Un servidor es un **sistema que proporciona recursos, datos, servicios o programas** a otros ordenadores (los "clientes") a través de una red.

Servidor proxy

Un proxy actúa como **intermediario entre emisor y receptor**, ocultando la IP real del usuario y aportando anonimato en ambas direcciones. No debe confundirse con una VPN, que en teoría es más segura.

Ventajas del proxy en red interna:

- Menor tiempo de configuración (solo se configura el proxy).
- Mayor seguridad y filtrados más eficientes.
- Mayor velocidad.
- **Ventaja principal: el anonimato.**

Desventajas:

- **Sobrecarga:** muchas peticiones simultáneas pueden saturarlo.
- **Caché de datos:** puede guardar copias de transferencias (riesgo de intromisión).
- **Desactualización:** la información más reciente puede verse afectada.

Tipos de proxy

- **Proxy web:** El usuario accede manualmente vía navegador; el servidor HTTP recibe la petición y devuelve el contenido.
- **Proxy inverso:** Se sitúa delante de los servidores web y reenvía las solicitudes de los clientes hacia ellos.
- **Proxy NAT:** Reescribe direcciones IP fuente/destino de los paquetes (enmascaramiento).
- **Proxy interdominio:** Usado por tecnologías asíncronas (Flash, AJAX, Comet) con restricciones entre dominios.
- **Proxy abierto:** Ejecuta cualquier petición de cualquier ordenador que se conecte a él.

El mapa de red de una compañía es un **dato superconfidencial**: usar un proxy por delante evita exponer las IPs internas (bases de datos, servidores, HSMs).

Nginx

Nginx es un servidor web de código abierto y gratuito, creado por Igor Sysoev en octubre de 2004 para resolver el "problema C10K" (gestionar 10.000 peticiones concurrentes). Además de servidor web, ofrece proxy inverso, balanceo de carga y caché HTTP. Lo usan compañías como Google, GitLab, VMWare, LinkedIn, Facebook o Twitter.

Características técnicas: proxy inverso con caché, soporte IPv6, balanceo de carga, FastCGI con caché, WebSockets, manejo de archivos estáticos/índice/autoindexación, y TLS/SSL.

Nginx vs. Apache

Criterio	Resultado
Compatibilidad SO	Ambos funcionan en Linux; en Windows, Apache rinde mejor que Nginx.
Soporte al usuario	Amplia documentación en ambos; Nginx destaca por su comunidad activa (Apache no tiene)
Rendimiento	Nginx es hasta 2 veces más rápido, usa menos memoria y es mejor para páginas estáticas;

2. Servidor de web

Un servidor web es el **contenedor donde se despliegan las páginas web** visibles desde el navegador, siguiendo el modelo cliente-servidor. La comunicación se basa en **HTTP** (hypertext transfer protocol); su variante cifrada **HTTPS** protege la confidencialidad de los datos.

Las peticiones más comunes son **GET** (solicitan información: HTML, XML, fotos, vídeos) y **POST** (envían datos al servidor, por ejemplo un formulario).

Servidores de aplicaciones más conocidos

- **Apache:** Software multiplataforma (Unix/Windows) que conecta el servidor con los navegadores, transfiriendo archivos entre ellos.
- **Tomcat:** Desarrollado por Apache Software Foundation; servidor HTTP orientado a aplicaciones Java (Servlet, JSP, Java EL, WebSocket). Menos eficiente que Apache para páginas estáticas.
- **Microsoft IIS:** Servidor web y conjunto de servicios (FTP, SMTP, NNTP, HTTP/HTTPS) para Windows; convierte un PC en servidor web local o remoto.
- **WebLogic (Oracle):** Servidor de aplicaciones Java EE y servidor web HTTP; compatible con Oracle, DB2, SQL Server y estándares JDBC/WS-Security.
- **WebSphere (IBM):** Familia de software middleware privado, referida popularmente al WebSphere Application Server (WAS); integra procesos de negocio en aplicaciones e-business.

3. Kubernetes

También llamado **k8s**, es una **plataforma portable y extensible de código abierto** desarrollada por ingenieros de Google para administrar cargas de trabajo y servicios en contenedores. Su origen se remonta a **Borg**, la plataforma interna de Google que gestiona más de 2.000 millones de implementaciones semanales.

Su clave: facilita la automatización y la configuración declarativa, eliminando procesos manuales en el despliegue y escalabilidad de aplicaciones en contenedores.

Conceptos fundamentales

- **Master:** Máquina que controla los nodos; origina todas las asignaciones de tareas.
- **Nodo:** Máquina que ejecuta las tareas asignadas, controlada por el master.
- **Pod:** Grupo de uno o más contenedores en un nodo único; comparten IP, IPC y host.
- **Controlador de replicación:** Controla cuántas copias idénticas de un pod deben ejecutarse en el clúster.
- **Servicio:** Separa las tareas de los pods; enruta las solicitudes automáticamente al pod correspondiente.
- **Kubelet:** Servicio en cada nodo que lee los manifiestos y garantiza que los contenedores se ejecuten.
- **Kubecttl:** Herramienta de línea de comandos para configurar Kubernetes.

Principales ventajas del uso de contenedores (según la documentación oficial)

- 1 Ágil creación y despliegue de aplicaciones.
- 2 Desarrollo, integración y despliegue continuo (rollbacks sencillos gracias a imágenes inmutables).
- 3 Separación de tareas entre Dev y Ops.
- 4 Observabilidad de la salud de la aplicación, no solo del sistema operativo.
- 5 Consistencia entre entornos de desarrollo, pruebas y producción.
- 6 Portabilidad entre nubes y distribuciones (Ubuntu, RHEL, CoreOS, GKE, etc.).
- 7 Administración centrada en la aplicación, no en el hardware.
- 8 Microservicios distribuidos, elásticos y débilmente acoplados.
- 9 Aislamiento de recursos para un rendimiento más predecible.
- 10 Mayor eficiencia y densidad en la utilización de recursos.

4. Patrones MVC

El patrón **MVC (Modelo-Vista-Controlador)** es una guía arquitectónica para organizar los componentes de un software, sus funciones y relaciones. Se originó en 1979 en el ámbito de las interfaces gráficas de usuario y sigue vigente hoy.

- **Modelo:** Representa la base del dominio y la lógica de negocio (p. ej. operaciones de conversión de moneda). También conocido como capa DTO; gestiona el almacenamiento y recuperación de datos.
- **Vista:** Genera la interfaz de la aplicación: botones, cuadros de edición y demás elementos que permiten al usuario enviar información y ordenar acciones.
- **Controlador:** Intermediario entre usuario y sistema: captura las acciones sobre la vista, las interpreta y actúa —consultando o actualizando el modelo, o mostrando una nueva vista. Funciona como "director de orquesta" del sistema.

Ventajas y desventajas del patrón MVC

Ventajas	Desventajas
----------	-------------

Sistemas más limpios y mantenibles gracias a la separación de responsabilidades	El patrón define las abstracciones y al patrón marcado por los componentes
Mayor velocidad de desarrollo: se puede trabajar en paralelo	El aprendizaje es sencillo
Múltiples vistas a partir del mismo modelo	
Gran facilidad para pruebas unitarias	

5. Conclusiones

El fastbook recorre los pilares de la arquitectura de aplicaciones web: servidores de proxy y de aplicaciones (destacando Tomcat y Apache para proyectos más pequeños, y WebLogic o WebSphere como opciones de pago para aplicaciones más grandes). Entre los orquestadores, **Kubernetes es hoy el más utilizado**, hasta el punto de que las grandes compañías migran sus aplicaciones a contenedores, y proveedores como AWS, Google Cloud o Azure ya lo ofrecen como servicio gestionado. Finalmente, el patrón **MVC** (modelo, vista, controlador) se mantiene como referencia clave para lograr aplicaciones más lógicas y homogéneas.

Resumen elaborado a partir de "Fastbook 03 — Arquitectura de aplicaciones web: conceptos clave" (Autor original: Jaime Morales).